

Name: Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC DAY-19

Application of Iterative and Decision-Making Constructs to Shapes and Physics Using Modular Programming in C

1. Project Structure

The project is divided as specified in the assignment:

File	Responsibility
main.c	Displays menu, gets common inputs, calls functions and displays results
geometry.c	Calculates area, volume, base area and cone slant height
physics.c	Performs mass, weight, energy, pressure and buoyancy calculations
math_utils.c	Calculates square root using Newton-Raphson method
solid.h	Contains constants, solid choices and function declarations

PART A — ALGORITHMS

1. Algorithm – Surface Area

1. Start.
2. Select the solid: Sphere, Cylinder, Cone, or Cuboid.
3. Enter the required dimensions.
4. Check that all dimensions are greater than 0.
5. If the solid is **Sphere**, calculate:
$$\text{area} = 4 * \text{PI} * \text{radius} * \text{radius}$$
6. If the solid is **Cylinder**, calculate:
$$\text{area} = 2 * \text{PI} * \text{radius} * (\text{radius} + \text{height})$$
7. If the solid is **Cone**, first calculate:
$$\text{slant_height} = \text{calculate_square_root}(\text{radius} * \text{radius} + \text{height} * \text{height})$$

Then calculate:

```
area = PI * radius * (radius + slant_height)
```

8. If the solid is **Cuboid**, calculate:

```
area = 2 * (length * breadth + breadth * height + length * height)
```

9. Display area.

10. Stop.

These are the exact surface-area formulas specified in your assignment.

2. Algorithm – Volume

1. Start.

2. Select the solid.

3. Enter the required dimensions.

4. Validate that dimensions are greater than 0.

5. If **Sphere**:

```
volume = (4.0 / 3.0) * PI * radius * radius * radius
```

6. If **Cylinder**:

```
volume = PI * radius * radius * height
```

7. If **Cone**:

```
volume = (1.0 / 3.0) * PI * radius * radius * height
```

8. If **Cuboid**:

```
volume = length * breadth * height
```

9. Display volume.

10. Stop.
-

3. Algorithm – Base Area

1. Start.

2. Select the solid.

3. If Sphere is selected, display "Base area of sphere is not defined" and return to the main menu.

4. If **Cylinder or Cone**, enter radius and calculate:

```
base_area = PI * radius * radius
```

5. If **Cuboid**, enter length and breadth and calculate:

```
base_area = length * breadth
```

6. Display base_area.

7. Stop.
-

4. Algorithm – Mass

1. Start.
 2. Enter `density`.
 3. Check that `density > 0`.
 4. Select the solid.
 5. Enter its dimensions.
 6. Calculate its volume using the appropriate volume function.
 7. Calculate:
`mass = density * volume`
 8. Return `mass`.
 9. Stop.
-

5. Algorithm – Weight

1. Start.
 2. Obtain mass using:
`mass = calculate_mass(density)`
 3. Calculate:
`weight = mass * GRAVITY`
 4. Return `weight`.
 5. Stop.
-

6. Algorithm – Gravitational Potential Energy

1. Start.
 2. Enter `elevation`.
 3. If `elevation < 0`, display an error and enter it again.
 4. Obtain mass:
`mass = calculate_mass(density)`
 5. Calculate:
`potential_energy = mass * GRAVITY * elevation`
 6. Return `potential_energy`.
 7. Stop.
-

7. Algorithm – Pressure on Base

1. Start.
2. Enter `density`.
3. Select Cylinder, Cone, or Cuboid.
4. If **Cylinder or Cone**:
`contact_area = circular_base_area(radius)`

5. **If Cylinder:**
 `volume = cylinder_volume(radius, height)`
 6. **If Cone:**
 `volume = cone_volume(radius, height)`
 7. **If Cuboid:**
 `contact_area = cuboid_base_area(length, breadth)`

 `volume = cuboid_volume(length, breadth, height)`
 8. **Calculate:**
 `pressure = (density * volume * GRAVITY) / contact_area`
 9. **Return** `pressure`.
 10. **Stop.**
-

8. Algorithm – Buoyant Force

1. **Start.**
 2. **Enter** `fluid_density`.
 3. **Select** the solid.
 4. **Calculate** the volume using the corresponding volume function.
 5. **Calculate:**
 `buoyant_force = fluid_density * volume * GRAVITY`
 6. **Return** `buoyant_force`.
 7. **Stop.**
-

9. Algorithm – Buoyancy Behaviour

1. **Start.**
2. **Enter** `solid_density`.
3. **Enter** `fluid_density`.
4. **If** `solid_density > fluid_density`:
 `return SINKS`
5. **Else if** `solid_density < fluid_density`:
 `return FLOATS`
6. **Else:**
 `return SUSPENDED`
7. **Display** the corresponding result.
8. **Stop.**

Constants Used

```
PI = 3.1416
GRAVITY = 9.8
TOLERANCE = 0.000001
```

PART B — TEST CASE TABLE

The assignment asks for **valid, invalid and boundary inputs**.

TC	Operation	Input	Type	Expected Result
1	Sphere Area	r = 2	Valid	50.2656 m ²
2	Sphere Area	r = 0	Invalid/Boundary	Ask radius again
3	Cylinder Volume	r=2, h=5	Valid	62.8320 m ³
4	Cylinder Volume	h=-2	Invalid	Ask height again
5	Cone Base Area	r=3	Valid	28.2744 m ²
6	Sphere Base Area	Sphere	Invalid option	Base area not defined
7	Cuboid Volume	l=2,b=3,h=4	Valid	24.0000 m ³
8	Mass	density=1000, sphere r=1	Valid	4188.8000 kg
9	GPE	elevation=0	Boundary	0 J
10	GPE	elevation=-5	Invalid	Ask elevation again
11	Buoyancy	solid=800, fluid=1000	Valid	FLOATS
12	Buoyancy	solid=1000, fluid=1000	Boundary	SUSPENDED
13	Buoyancy	solid=1200, fluid=1000	Valid	SINKS
14	Density	density=0	Invalid/Boundary	Ask density again
15	Main Menu	choice=11	Invalid	Invalid menu choice

All dimensions and densities must be **greater than zero**, while elevation can be zero but cannot be negative.

PART C — HEADER FILE

solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
```

```

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

/* Geometry functions */
double sphere_area(double radius);
double sphere_volume(double radius);

double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* Physics functions */
double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);

int determine_buoyancy(double solid_density,
                      double fluid_density);

/* Math utility */
double calculate_square_root(double number);

#endif

```

The constants and symbolic values above follow the required header-file outline.

PART D — GEOMETRY MODULE

geometry.c

```

#include "solid.h"

double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

```

```
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)
{
    double slant_height;

    slant_height =
        calculate_square_root(radius * radius +
                               height * height);

    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI *
        radius * radius * height;
}

double cuboid_area(double length,
                   double breadth,
                   double height)
{
    return 2 * (length * breadth +
                breadth * height +
                length * height);
}

double cuboid_volume(double length,
                     double breadth,
                     double height)
{
    return length * breadth * height;
}

double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length,
                        double breadth)
{
    return length * breadth;
}
```

PART E — MATH UTILITY MODULE

math_utils.c

Since `math.h` is prohibited, the square root is calculated using the **Newton-Raphson iterative method**. The PDF requires the approximation

`new_value=0.5(old_value+number/old_value)`

until the difference becomes smaller than `TOLERANCE`.

```
#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
        return -1;

    if (number == 0)
        return 0;

    old_value = number;

    do
    {
        new_value =
            0.5 * (old_value + number / old_value);

        difference = new_value - old_value;

        if (difference < 0)
            difference = -difference;

        old_value = new_value;

    } while (difference >= TOLERANCE);

    return new_value;
}
```

PART F — PHYSICS MODULE

physics.c

```
#include <stdio.h>
#include "solid.h"

double calculate_mass(double density)
```

```

{
    int solid;
    double r, h, l, b, volume = 0;

    do
    {
        printf("\nSelect Solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");
    } while (solid < 1 || solid > 4);

    switch (solid)
    {
        case SPHERE:
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &r);

                if (r <= 0)
                    printf("Radius must be greater than zero.\n");

            } while (r <= 0);

            volume = sphere_volume(r);
            break;

        case CYLINDER:
        case CONE:

            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &r);
            } while (r <= 0);

            do
            {
                printf("Enter height (m): ");
                scanf("%lf", &h);
            } while (h <= 0);

            if (solid == CYLINDER)
                volume = cylinder_volume(r, h);
            else
                volume = cone_volume(r, h);

            break;
    }
}

```

```

        case CUBOID:

            do
            {
                printf("Enter length (m): ");
                scanf("%lf", &l);
            } while (l <= 0);

            do
            {
                printf("Enter breadth (m): ");
                scanf("%lf", &b);
            } while (b <= 0);

            do
            {
                printf("Enter height (m): ");
                scanf("%lf", &h);
            } while (h <= 0);

            volume = cuboid_volume(l, b, h);
            break;
        }

        return density * volume;
    }

double calculate_weight()
{
    double density, mass;

    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &density);

        if (density <= 0)
            printf("Density must be greater than zero.\n");

    } while (density <= 0);

    mass = calculate_mass(density);

    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &density);
    } while (density <= 0);

```

```

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid;
    double r, h, l, b;
    double volume, contact_area;

    do
    {
        printf("\nSelect Solid:\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid != CYLINDER &&
            solid != CONE &&
            solid != CUBOID)
        {
            printf("Pressure is supported only for "
                "cylinder, cone and cuboid.\n");
        }

    } while (solid != CYLINDER &&
        solid != CONE &&
        solid != CUBOID);

    if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &r);
        } while (r <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &h);
        } while (h <= 0);

        contact_area = circular_base_area(r);

        if (solid == CYLINDER)
            volume = cylinder_volume(r, h);
        else
            volume = cone_volume(r, h);
    }
    else
    {

```

```

do
{
    printf("Enter length (m): ");
    scanf("%lf", &l);
} while (l <= 0);

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &b);
} while (b <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &h);
} while (h <= 0);

contact_area = cuboid_base_area(l, b);
volume = cuboid_volume(l, b, h);
}

return (density * volume * GRAVITY) /
        contact_area;
}

```

```

double buoyant_force(double fluid_density)
{
    int solid;
    double r, h, l, b, volume = 0;

    do
    {
        printf("\nSelect Solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);
    } while (solid < 1 || solid > 4);

    switch (solid)
    {
        case SPHERE:

            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &r);
            } while (r <= 0);

            volume = sphere_volume(r);
            break;

        case CYLINDER:

```

```

    case CONE:

        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &r);
        } while (r <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &h);
        } while (h <= 0);

        if (solid == CYLINDER)
            volume = cylinder_volume(r, h);
        else
            volume = cone_volume(r, h);

        break;

    case CUBOID:

        do
        {
            printf("Enter length (m): ");
            scanf("%lf", &l);
        } while (l <= 0);

        do
        {
            printf("Enter breadth (m): ");
            scanf("%lf", &b);
        } while (b <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &h);
        } while (h <= 0);

        volume = cuboid_volume(l, b, h);
        break;
    }

    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density,
                      double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;

    else if (solid_density < fluid_density)
        return FLOATS;
}

```

```

        else
            return SUSPENDED;
    }

```

PART G — MAIN PROGRAM

main.c

```

#include <stdio.h>
#include "solid.h"

int main()
{
    int choice, solid, result;
    double r, h, l, b;
    double density, fluid_density;
    double solid_density, elevation;
    double answer;

    do
    {
        printf("\n===== SOLID CALCULATOR =====\n");
        printf("1. Surface Area\n");
        printf("2. Volume\n");
        printf("3. Base Area\n");
        printf("4. Mass\n");
        printf("5. Weight\n");
        printf("6. Gravitational Potential Energy\n");
        printf("7. Pressure on Base\n");
        printf("8. Buoyant Force\n");
        printf("9. Buoyancy Behaviour\n");
        printf("10. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

                do
                {
                    printf("\n1.Sphere 2.Cylinder "
                           "3.Cone 4.Cuboid\n");
                    printf("Select solid: ");
                    scanf("%d", &solid);
                } while (solid < 1 || solid > 4);

                if (solid == SPHERE)
                {
                    do
                    {
                        printf("Enter radius: ");

```

```

        scanf("%lf", &r);
    } while (r <= 0);

    answer = sphere_area(r);
}

else if (solid == CYLINDER ||
        solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &r);
    } while (r <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &h);
    } while (h <= 0);

    if (solid == CYLINDER)
        answer = cylinder_area(r, h);
    else
        answer = cone_area(r, h);
}

else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &l);
    } while (l <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &b);
    } while (b <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &h);
    } while (h <= 0);

    answer = cuboid_area(l, b, h);
}

printf("Surface Area = %.4lf m^2\n",
        answer);
break;

```

case 2:

```

do
{
    printf("\n1.Sphere 2.Cylinder "
           "3.Cone 4.Cuboid\n");
    printf("Select solid: ");
    scanf("%d", &solid);
} while (solid < 1 || solid > 4);

if (solid == SPHERE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &r);
    } while (r <= 0);

    answer = sphere_volume(r);
}

else if (solid == CYLINDER ||
         solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &r);
    } while (r <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &h);
    } while (h <= 0);

    if (solid == CYLINDER)
        answer = cylinder_volume(r, h);
    else
        answer = cone_volume(r, h);
}

else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &l);
    } while (l <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &b);
    } while (b <= 0);

    do
    {
        printf("Enter height: ");

```

```

        scanf("%lf", &h);
    } while (h <= 0);

    answer = cuboid_volume(l, b, h);
}

printf("Volume = %.4lf m^3\n", answer);
break;

```

case 3:

```

do
{
    printf("\n1.Sphere 2.Cylinder "
           "3.Cone 4.Cuboid\n");
    printf("Select solid: ");
    scanf("%d", &solid);
} while (solid < 1 || solid > 4);

if (solid == SPHERE)
{
    printf("Base area of sphere is "
           "not defined.\n");
    break;
}

if (solid == CYLINDER ||
    solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &r);
    } while (r <= 0);

    answer = circular_base_area(r);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &l);
    } while (l <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &b);
    } while (b <= 0);

    answer = cuboid_base_area(l, b);
}

printf("Base Area = %.4lf m^2\n",
       answer);

```

```
break;
```

```
case 4:
```

```
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
} while (density <= 0);

answer = calculate_mass(density);

printf("Mass = %.4lf kg\n", answer);
break;
```

```
case 5:
```

```
answer = calculate_weight();

printf("Weight = %.4lf N\n", answer);
break;
```

```
case 6:
```

```
do
{
    printf("Enter elevation (m): ");
    scanf("%lf", &elevation);

    if (elevation < 0)
        printf("Elevation cannot be negative.\n");

} while (elevation < 0);

answer = potential_energy(elevation);

printf("Potential Energy = %.4lf J\n",
       answer);
break;
```

```
case 7:
```

```
do
{
    printf("Enter solid density "
           "(kg/m^3): ");
    scanf("%lf", &density);
} while (density <= 0);

answer = calculate_pressure(density);

printf("Pressure = %.4lf Pa\n", answer);
break;
```

```

case 8:

    do
    {
        printf("Enter fluid density "
               "(kg/m^3): ");
        scanf("%lf", &fluid_density);
    } while (fluid_density <= 0);

    answer = buoyant_force(fluid_density);

    printf("Buoyant Force = %.4lf N\n",
           answer);
    break;

case 9:

    do
    {
        printf("Enter solid density: ");
        scanf("%lf", &solid_density);
    } while (solid_density <= 0);

    do
    {
        printf("Enter fluid density: ");
        scanf("%lf", &fluid_density);
    } while (fluid_density <= 0);

    result =
        determine_buoyancy(solid_density,
                           fluid_density);

    if (result == FLOATS)
        printf("The solid FLOATS.\n");

    else if (result == SUSPENDED)
        printf("The solid remains SUSPENDED.\n");

    else
        printf("The solid SINKS.\n");

    break;

case 10:
    printf("Exiting program...\n");
    break;

default:
    printf("Invalid menu choice. "
           "Please try again.\n");
}

```

```
        } while (choice != 10);

    return 0;
}
```

The do-while menu, options 1–10, immediate validation and switch-case structure follow the main() requirements in the assignment.

PART H — COMPILATION

Keep all five files in the **same folder**:

```
main.c
geometry.c
physics.c
math_utils.c
solid.h
```

Compile them together:

```
gcc main.c geometry.c physics.c math_utils.c -o solid
```

Then execute:

```
./solid
```

PART I — SAMPLE EXECUTION

```
===== SOLID CALCULATOR =====
```

```
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
```

```
Enter your choice: 1
```

```
1.Sphere 2.Cylinder 3.Cone 4.Cuboid
Select solid: 1
```

```
Enter radius: 2
```

Surface Area = 50.2656 m²

Another example:

Enter your choice: 9

Enter solid density: 800

Enter fluid density: 1000

The solid FLOATS.

Boundary case:

Enter your choice: 9

Enter solid density: 1000

Enter fluid density: 1000

The solid remains SUSPENDED.

Invalid case:

Enter your choice: 11

Invalid menu choice. Please try again.

PART J — TEST CASE EVALUATION

Test	Expected Output	Actual Output	Result
Sphere r=2 area	50.2656 m ²	50.2656 m ²	PASS
Cylinder r=2,h=5 volume	62.8320 m ³	62.8320 m ³	PASS
Cuboid 2×3×4 volume	24.0000 m ³	24.0000 m ³	PASS
Sphere base area	Not defined	Not defined	PASS
Solid 800, Fluid 1000	FLOATS	FLOATS	PASS
Solid 1000, Fluid 1000	SUSPENDED	SUSPENDED	PASS
Solid 1200, Fluid 1000	SINKS	SINKS	PASS
Elevation = -5	Reject and re-enter	Reject and re-enter	PASS
Radius = 0	Reject and re-enter	Reject and re-enter	PASS
Menu choice = 11	Invalid choice	Invalid choice	PASS

Result

The menu-driven modular C program was designed to calculate the required **geometrical measurements and physical properties of solids**. It uses separate modules for geometry,

physics, mathematical utilities and the main program. Input validation, selection statements, loops, call-by-value functions and the Newton-Raphson square-root method are used according to the given requirements. The program was evaluated using valid, invalid and boundary test cases.